

Group Name: Player

Project 2 Documentation

Name of the player program: PlayerGemini(src/main/java/PlayerGemini.java)

Additional files/program:

- State.java: Store function and attributes for board configurations
- GameConstants.java: Store constants and dictionaries for the game
- Gemini.bat: Store the command to run the GeminiPlayer
- Minimax.bat: Store the command to run the Player (from Minimax)
- config.env: Store the API Key for Gemini API (the submission key is Quan's API key)
- pom.xml: Store the dependencies for the project

I. Members of the Group

Quan Dinh

- Implemented the base data-structure for the Lasker Morris game: State's attributes, mill, and neighbor dictionaries
- Integrated Gemini API with the foundational code, and set rate-limit control
- Prompt Engineering with Gemini
- Bug Fixing

Elijah Gray

- Implemented the functions for updating move receive from referee and move receive from Gemini to the stored data states.
- Prompt Engineering with Gemini
- Bug Fixing
- Move Validation Code
- Testing

Joseph Pisano

- Wrote the functions for checking the validity of moves from both the opponent and the ones generated by the player to make sure they are legal moves.
- General Bug Fixing
- Prompt Engineering with Gemini.
- Wrote documentation.

II. Instructions for Compilation

Use any java compiler with Maven installed with the GeminiPlayer.java file:

1. Integrate Maven with the compiler (i.e IntelliJ)
2. Run '**mvn clean install**' in terminal (*Must have Maven and Java version 18+*)
3. Run the file GeminiPlayer.java in the compiler

4. The program will start and the user can interact with the program through the terminal. The user can type “orange” or “blue” to tell the program which stone it should be.
5. The user can interact with the program to play Lasker Morris (by entering the specified move format)

To run the program with referee:

1. Run ‘**mvn clean install**’ in terminal (*Must have Maven and Java version 18+*)
2. Then run ‘**mvn compile**’
3. Enter “**cs4341-referee laskermorris -l -t 60 -p1 "Gemini.bat" -p2 "Minimax.bat" --visual**” in terminal to run the referee.
4. The referee will call one instance of the Gemini program and another of the original Minimax from project 1 to play against each other.
5. Once the game ends, the referee will return the result of the game.

Note:

Make sure you have the pom.xml file at root in order to get all the dependencies. We use .bat to compile the Maven program since the series of commands generated by the program at execution print out data that the referee mistakenly read. This allows us to also run the program much faster than orthodox commands.

!!! This means that you can replace the second line of run.txt with Gemini.bat

III. System Description

In development, we each configured our own Gemini API key in order to maximize the testing limits of Gemini AI. This key is stored locally in config.env.

The Gemini API was integrated via importing it into our Java program using and compiling using Maven. From there we supplied it the API key and would send the prompt via our written getGeminiMoves function, which would take the client with the API key and compile all of the current state information along with a pre-written prompt to then send to Gemini. Gemini was instructed to only supply the move in the format compatible with the referee, meaning no parsing of the response was needed.

Upon receiving the move Gemini had decided on the function checkLegalMove() would be used to check that Gemini had made a legal move. If Gemini didn’t, which happened occasionally as the LLM likes to cheat, it would be re-prompted with the original prompt along with what their error was in their previous move.

Gemini would be prompted at most 5 times per turn if there is an error, but it only gets prompted 1 time per turn if it runs successfully. We set the time in between each request to be 4 seconds at least in order to not exhaust the limit per minute.

Note: If grader decides to run Gemini against Gemini, you should reduce the maxTries or increase the timeLimits in the global variable of PlayerGemini.java. Otherwise, the rates will be exhausted and cause the program to only send back random integrated moves.

IV. Prompt Engineering

The final prompt used is located at the bottom of this file, as it is very long. The general layout of the prompt was as follows:

- Described that it is playing Lasker Morris and explained what its Stone pieces were represented by and what the Opponents were.
- Explained the Rules of the Game to it (Placing, moving, flying, Mill conditions, and capturing stones)
- Told it the Game State, including: the current board configuration, the stones it has in hand, the stones on the board, the opponents stones, empty locations on the board, and the last made move by the opponent.
- A description of the basic strategy used while playing Lasker Morris
- How to output it's response

From experimentation, we noticed that:

- Gemini has no understanding of what Lasker Morris is on its own. It cannot infer any of the rules despite the fact you might expect an LLM to know a little bit about everything. So we had to clearly describe every single rule.
- Gemini had a lot of trouble understanding it cannot place pieces on top of a pre-existing piece. This rule took substantial editing to improve its understanding. In general the LLM would struggle with understanding rules and break them frequently, which is why the checking of legal moves was a necessity.
- Gemini has no concept of strategy. For the most part it would just obey moves by coming up with a random, usually legal one. So in the prompt we had to explain some of the advantageous moves to look for in Lasker Morris for it to actually be able to make smarter moves. This helped a decent bit and helped it with playing mildly smarter, though still not at an average Player's capability.
- Gemini forgets to take a stone when it forms a mill. When it does form a mill, it often forgets to take a stone and therefore needs to be re-prompted to select an opponent's stone to be taken away.

V. Testing & Results

TESTINGS

The program was tested against human players and against the minimax algorithm from project 1

1. Playing against humans.
 - Variable mileage. Playing against someone who understands the game usually will result in Gemini losing. However when tested against someone who doesn't understand the game it's a fairly even chance, though that is mainly just due to someone having to end up the victor.
2. Playing against the minimax algorithm
 - The Gemini program loses every single time. The minimax algorithm is both faster and more optimal compared to Gemini.
3. Playing against the Gemini algorithm (itself)
 - The programs tend to draw because it seems to not go for any mill-closing or mill-blocking moves right away, which lead to the referee exiting early.

Can a LLM make a good game-playing agent?

- No. A LLM is very generalized and therefore it doesn't perform well as a game-playing agent. Especially in heavy strategy games, it is incapable of making smart decisions. It was even bad at simply following instructions most of the time, since there are too many conditions for it to memorize. The language used in prompting had to be incredibly strict for it to not just ignore them. A good game-playing agent is characterized by its specialization at the game, which is what makes a generalized AI like a LLM a bad choice for trying to have it play a game.
- However, if LLM is to be used as a "solely generative" game-playing and not exactly smart game-playing, then it is feasible. It is capable of generating legal moves and fixing its own mistakes if pointed out.

Final Prompt:

This is an example of a prompt that is sent through the API. The prompt changed several attributes within it dynamically with the board state

You are playing Lasker Morris according to the current board configuration. It is your turn.
You must make a valid move according to ALL the rules.

Your stone is: B

Your opponent's stone is: O

Your hand is represented by: h1

Game Rules:

Board:

- Sparse grid, x-axis: a-g, y-axis: 1-7.
- Locations: (number-letter) format (e.g., a1).

Basic Restrictions:

- Move/place only your stones.
- Target location must be empty.
- Start with 10 stones in hand.
- If no stones in hand, move a stone on the board (but you can move stone on board even when you have stone in hand).

Turn Actions:

- Place a stone from hand (your stone type) to an EMPTY location.
- Move a stone on the board: to an EMPTY adjacent location.
 - Adjacent locations: e5: d5, e4

b4: a4, b2, b6, c4

b6: b4, d6

g1: d1, g4

d1: a1, g1

d2: b2, f2, d1, d3

d3: c3, d2, e3

a1: a4, d1

g4: f4, g1, g7

d5: c5, d6, e5

a4: a7, b4

d6: b6, d7, d5, f6

d7: a7, g7

a7: a4, d7

g7: d7, g4

f2: d2, f4

f4: e4, f2, f6, g4

c3: c4, d3

c4: b4, c3, c5

c5: c4, d5

f6: d6, f4

e3: d3, e4

e4: e3, e5, f4

b2: d2, b4

- After placing/moving, ALWAYS check if you have formed a mill. If your action led you to form a mill this turn, then you MUST take action regarding removing an opponent's piece (check Mill Removals rules below)

Flying Phase:

- If your total stones from your (board + hand) == 3, you can move any of your stone to any empty location.

Mills Conditions:

- Three of your stones contiguously adjacent to each other form a mill.

- Possible mills:

[a7, d7, g7]

[b6, d6, f6]

[c5, d5, e5]

[a4, b4, c4]

[e4, f4, g4]

[c3, d3, e3]

[b2, d2, f2]

[a1, d1, g1]

[a1, a4, a7]

[b2, b4, b6]

[c3, c4, c5]

[d1, d2, d3]

[d5, d6, d7]

[e3, e4, e5]

[f2, f4, f6]

[g1, g4, g7]

- For example: If you have a stone on `a1` and `d1`, and you plan to place a stone to `g1`, that means you have formed a mill.

Current Board:

g1 = EMPTY

f2 = EMPTY

g4 = EMPTY

d1 = EMPTY

f4 = EMPTY

e3 = EMPTY

d2 = EMPTY

e4 = EMPTY

d3 = EMPTY

g7 = EMPTY
f6 = EMPTY
e5 = EMPTY
c3 = EMPTY
b2 = EMPTY
a1 = EMPTY
d5 = EMPTY
c4 = EMPTY
d6 = EMPTY
c5 = EMPTY
b4 = EMPTY
d7 = EMPTY
a4 = EMPTY
b6 = EMPTY
a7 = EMPTY

The board format is: location = Type of stone on that location.

If the location = EMPTY, then there is no stone on that location.

If the location = Your stone type, then you have a stone on that location.

If the location = Opponent's stone type, then opponent has a stone on that location.

The move that you are about to do will be updated to this board, so ALWAYS consider your own move this turn as if it is updated on here.

Game State:

- The number of your stones in hand: 10
- The number of your stones on board: 0
- The number of opponent's stones in hand: 10
- The number of opponent's stones on board: 0
- Empty locations on the board: [a7, d7, g7, b6, d6, f6, c5, d5, e5, a4, b4, c4, e4, f4, g4, c3, d3, e3, b2, d2, f2, a1, d1, g1]
- The last move that was made by your opponent is:
- Remember that the move you are calculating for this turn will update this board state, so check if the move you about to do will lead you to form a mill or not.

Mill Removal:

- If you form a mill this turn, you MUST ABSOLUTELY remove ONE opponent's stone from the board.
- Remove an opponent's stone (e.g., a1).
- Do not remove opponent's stones from their mills; unless all opponent stones are in mills, then remove any of their stones.

- If no opponent stones, do not remove.
- Always check if the move you are about to take will lead you to form a mill.

Win Conditions:

- Opponent has 2 or fewer stones.
- Opponent is immobilized (Opponent cannot move any of their stones and they cannot place anymore stones).

Strategy:

1. Prioritized Mill Check: After EACH of your potential moves, perform the following steps:
 - a. Temporarily update the current board state as if the move has been made.
 - b. Implement the following algorithm to check for mills:
 - i. Iterate through EACH of the possible mill formations (provided in 'Mills Conditions').
 - ii. For EACH mill formation, check if ALL three positions in that formation are now occupied by your stones in the *simulated* board state.
 - iii. If a mill is found, record the mill formations and proceed to step 2.
 - c. Undo the temporary board state update.
 2. Mill Closure (If Mill Formed): If a mill was formed in step 1, select an opponent's stone to remove (according to 'Mill Removal' rules).
 3. Move Selection:
 - a. Game-Ending Moves: If there are any moves that will lead you to immediately win the game, such as immobilizing ALL of your opponent's stones or reduces ALL of their total stones to 2, then ALWAYS go for this move.
 - b. Mill-Closing Moves: If there are any moves that create a mill (identified in step 1), always prioritize those moves over any other moves (except for Game-Ending Moves).
 - c. Other Moves: If no mill-closing moves are available, consider other strategic moves (blocking, setting up potential mills, etc.).
 4. Output: Provide the move in the specified format. Ensure <z1> is correctly set to the removed stone (if a mill was formed) or "r0" (if no mill).
- It is important that your move should always prioritized forming a mill this turn over setting up. Meaning you should form a mill at any chance you get.

Output Format:

- <x1> <y1> <z1>
- <x1>: place from hand (h1 if you are B, h2 if you are O) or stone location (move).
- <y1>: Empty target location.
- <z1>: Location of opponent's stone to remove (if mill), or r0 (no mill).
- Ensure <z1> is r0 ONLY if NO mill was formed this turn. r0 is not a location on the board and is just a placeholder to signify not removing any stones

- Recheck your <y1> to make sure that it is empty (Check the Game State: <y1> must be within that list)
- Recheck your <x1> and <y1> move against the current board and the mill conditions to see if you have created a mill

Examples moves:

- Move: a7 a4 r0 (Standard move: moving your stone from a7 to a4)
- Move: h1 a4 r0 (Placing from hand to a4)
- Move: c1 c4 a1 (Forming a mill by moving your stone from c1 to c4, and removing opponent piece at a1)

Provide your move in the specified format. Do not say anything more than just the formatted answer.